OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 3 – CO3

Name: R. Peeran Basha

Register Number: 192472244

Department / Class: CSE(AI)

Course: Object Oriented Programming with C++

Faculty: Jagadeesh K

Q1 – Constructors and Constructor Overloading

Objective: Create Book objects safely for an empty record, a donated book with a title, and a fully specified book.

Design: All constructors use member initializer lists. The default and title-only constructors delegate to the main parameterised constructor, so validation is centralised. A negative copy count is rejected with std::invalid_argument; this prevents an invalid Book object from being constructed.

Compiler rule: The compiler supplies a default constructor only when no user-declared constructor exists. Once a constructor is declared, a no-argument constructor is not automatically generated. Member initialisation follows the order of declaration in the class, not the order written in the initializer list. Initializer lists directly initialise members and are required for const/reference members; they also avoid default-construct-then-assign.

Object states: (1) defaultBook → title empty, author empty, copies 0; (2) donatedBook → title 'Clean Code', author empty, copies 1; (3) fullBook → title 'C++ Primer', author 'Lippman', copies 5.

Experimental/design notes: Removing the default constructor makes `Book shelf[3];` fail because each array element needs a default constructor. A wrong initializer order such as `Book(int x) : second(x), first(second) {}` when `first` is declared before `second` can trigger `-Wreorder` and may read an uninitialised member. `explicit` prevents unintended implicit conversions, e.g. `Book b = 3;` when a single-argument `Book(int)` constructor exists; with `explicit`, use `Book b(3)` or `Book b{3}`.

Alternative: One constructor with default arguments is possible, but several overloads communicate the supported creation cases more clearly. Both approaches create an object in O(1) time and O(1) auxiliary space. Delegation plus one validation point reduces duplicated checks.

C++17 Source Code

```
#include <iostream>
#include <string>
#include <stdexcept>
class Book {
    std::string title;
    std::string author;
    int copies;
public:
    Book() : Book("", "", 0) {}
    explicit Book(const std::string& t) : Book(t, "", 1) {}
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(c) {
        if (c < 0) throw std::invalid_argument("copies cannot be negative");
    }
    void print() const {
        std::cout << "Title: " << (title.empty() ? "(empty)" : title)
                  << ", Author: " << (author.empty() ? "(not set)" : author)
                  << ", Copies: " << copies << '\\n';
    }
};
int main() {
    try {
        Book emptyBook;
        Book donatedBook("Clean Code");
        Book fullBook("C++ Primer", "Lippman", 5);
        emptyBook.print();
```

```
        donatedBook.print();
        fullBook.print();
        // Book invalid("Broken Book", "Unknown", -2); // throws
    } catch (const std::invalid_argument& e) {
        std::cout << "Error: " << e.what() << '\\n';
    }
}
```

Expected Output

```
Title: (empty), Author: (not set), Copies: 0
Title: Clean Code, Author: (not set), Copies: 1
Title: C++ Primer, Author: Lippman, Copies: 5
```

Q2 – Copy Constructors: Shallow and Deep Copies

Objective: Ensure each Prescription owns an independent dynamically allocated medicine-name array.

Shallow copy problem: The compiler-generated copy constructor copies the pointer value, not the pointed-to array. Two objects then refer to the same allocation. A change through either object affects both, and both destructors attempt to delete the same allocation, causing double deletion/undefined behaviour.

Deep copy: Allocate a fresh array of the same length and copy each string. Copy assignment first creates a new allocation, copies the source, then releases the old allocation and adopts the new one. This ordering also handles self-assignment safely when guarded explicitly.

Rule of Three: If a class manually manages a resource and needs a destructor, it usually needs a copy constructor and copy assignment operator as well. Copy construction occurs when creating an object from another object, passing an object by value, and returning an object by value (though copy elision may remove some copies). Passing by const reference does not copy the object.

Memory view: Shallow: p.medicines ⟍ and q.medicines ⟋ same array. Deep: p.medicines → array A; q.medicines → independent array B.

Complexity: Copying n medicine strings takes O(n) element copies and O(n) extra storage (excluding the characters' own lengths). A shallow pointer copy is O(1) but unsafe for unique ownership. With std::vector<std::string>, compiler-generated copy operations correctly copy the vector's contents (Rule of Zero).

Required evidence: Run both the intentionally shallow version and the deep-copy version under AddressSanitizer. The unsafe version may report double-free/use-after-free; exact diagnostic text depends on compiler/runtime and is not fabricated here. Also test p = p and record timings for 10, 1,000 and 100,000 medicines.

C++17 Source Code

```
#include <iostream>
#include <string>
#include <algorithm>
class Prescription {
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new std::string[count] : nullptr) {}
    Prescription(const Prescription& other)
        : n(other.n), medicines(n > 0 ? new std::string[n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
    }
    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;
        std::string* fresh = other.n > 0
            ? new std::string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
```

```
    }
    ~Prescription() { delete[] medicines; }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    r = r; // self-assignment test
    std::cout << p.get(0) << ' ' << p.get(1) << " | "
              << q.get(0) << ' ' << q.get(1) << " | "
              << r.get(0) << ' ' << r.get(1) << '\\n';
}
```

Expected Output

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

Q3 – Dynamic Constructors and Destructors

Objective: Allocate a two-dimensional auditorium seat map dynamically and release every allocation when the Event object is destroyed.

Each row is allocated separately with `new char[seats]`; the row-pointer array is allocated with `new char*[rows]`. Destruction must use `delete[]` for every row and then `delete[]` for the row-pointer array. Using scalar `delete` for an array is undefined behaviour. Forgetting delete leaks memory.

Lifetime order: automatic local objects are destroyed in reverse order of construction when their scope ends. Objects inside a nested block are destroyed at the closing brace. A dynamically allocated object is not destroyed until `delete` is called. A destructor has no parameters, has no return type and cannot be overloaded because destruction has one fixed role for a class.

RAII: tying resource release to object lifetime ensures cleanup happens automatically on normal scope exit and during stack unwinding. The zero-row case is handled without allocating a row map.

Alternative layout: one contiguous allocation for rows × seats needs one allocation rather than rows + 1 allocations, and uses O(1) extra row-pointer storage. Access is O(1) in either design. A vector-of-vectors is safer and manages memory automatically, but may use separate row allocations.

Memory estimate for a char seat map: 10×20 = 200 bytes; 100×200 = 20,000 bytes; 1,000×2,000 = 2,000,000 bytes, excluding pointer-array and allocator overhead.

C++17 Source Code

```
#include <iostream>
#include <string>
class Event {
    int rows, seats, booked = 0;
    char** map;
public:
    Event(int r, int s) : rows(r), seats(s), map(nullptr) {
        if (rows <= 0 || seats <= 0) { rows = seats = 0; }
        else {
            map = new char*[rows];
            for (int i = 0; i < rows; ++i) {
                map[i] = new char[seats];
                for (int j = 0; j < seats; ++j) map[i][j] = 'O';
            }
        }
        std::cout << "Event opened: " << rows << "x" << seats << '\\n';
    }
    bool book(int r, int s) {
        if (!map || r < 0 || r >= rows || s < 0 || s >= seats ||
            map[r][s] == 'X') return false;
        map[r][s] = 'X'; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (!map || r < 0 || r >= rows || s < 0 || s >= seats ||
            map[r][s] != 'X') return false;
        map[r][s] = 'O'; --booked; return true;
    }
    ~Event() {
        if (map) {
            for (int i = 0; i < rows; ++i) delete[] map[i];
            delete[] map;
```

```
        }
        std::cout << "Event closed. Booked seats at closing: "
                  << booked << '\\n';
    }
};
int main() {
    Event local(2, 3);
    local.book(0, 1);
    {
        Event blockEvent(1, 2);
        blockEvent.book(0, 0);
    }
    Event* heapEvent = new Event(1, 2);
    heapEvent->book(0, 1);
    delete heapEvent;
    Event emptyEvent(0, 5);
}
```

Expected Output

```
Event opened: 2x3
Event opened: 1x2
Event closed. Booked seats at closing: 1
Event opened: 1x2
Event closed. Booked seats at closing: 1
Event opened: 0x0
Event closed. Booked seats at closing: 0
Event closed. Booked seats at closing: 1
```

Q3 – Predict the Output (Logger)

Prediction: constructing `a` prints +A; constructing `b` prints +B and its destructor prints -B at the end of the nested block. Passing `a` by value to `use(Logger)` invokes the copy constructor, creating `Ac`; the function prints `use`, then the parameter is destroyed and prints -Ac. The heap object prints +P and is destroyed by delete, printing -P. `end` is printed before `a` is destroyed at the end of main.

Constructor/destructor order follows object lifetime. The copy exists only for the by-value parameter. If the parameter were `const Logger&`, this particular copy would not be required.

C++17 Source Code

```
#include <iostream>
#include <string>
struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") {
        std::cout << "copy" << n << ' ';
    }
    ~Logger() { std::cout << '-' << n << ' '; }
};
void use(Logger) { std::cout << "use "; }
int main() {
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    std::cout << "end ";
}
```

Expected Output

```
+A +B -B copyAc use -Ac +P -P end -A
```

Q4 – Overloading Arithmetic and Stream Operators

Objective: Represent money exactly as an integer number of paise and make billing expressions readable.

`+` returns a new Money value and does not modify its operands. `+=` modifies the left operand and returns `Money&` to `*this`, enabling chaining. `==` and `<` compare the stored paise. Stream insertion `<<` must be a non-member because the left operand is `std::ostream`; it is declared friend to access private paise. `a + b` is translated conceptually to `a.operator+(b)` for a member overload.

Negative subtraction: this implementation permits negative Money values to represent a credit or refund. A production billing system may instead reject negative balances or model refunds explicitly.

Engineering decision: multiplying one Money amount by another Money amount is generally dimensionally meaningless. Multiplication by a scalar quantity (e.g. item count or tax rate) may be useful, with explicit rounding rules.

Alternative: friend non-member arithmetic overloads can support symmetric conversions such as `100 + m` and `m + 100`; a member overload naturally has the left operand as the class object. Named functions add()/equals() are also O(1). All these forms use O(1) time and O(1) object space. Integer paise avoids binary floating-point representation errors.

C++17 Source Code

```
#include <iomanip>
#include <iostream>
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }
    Money& operator+=(const Money& other) {
        paise += other.paise;
        return *this;
    }
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }
    bool operator<(const Money& other) const {
        return paise < other.paise;
    }
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long magnitude = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) os << '-';
        os << "Rs " << magnitude / 100 << '.'
           << std::setw(2) << std::setfill('0') << magnitude % 100
           << std::setfill(' ');
        return os;
    }
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
```

```
    std::cout << c << '\\n' << std::boolalpha << (c == Money(200)) << '\\n';
    Money carry(0, 75), extra(0, 50);
    std::cout << (carry + extra) << '\\n';
    Money refund(1, 0);
    refund += Money(-2, 0);
    std::cout << refund << '\\n';
}
```

Expected Output

```
Rs 200.00
true
Rs 1.25
-Rs 1.00
```

Q5 – Overloading Increment, Comparison and Subscript Operators

Objective: Implement prefix/postfix increment for Time and checked indexing for a Playlist.

Prefix `++t` has no dummy parameter and returns a reference to the updated object. Postfix `t++` has an unused int parameter to distinguish it; it returns the old value by copy, then increments the object. Postfix therefore creates a temporary copy and is often less efficient for class types.

Playlist has two `operator[]` overloads: a non-const version returning `std::string&` so callers can modify a track, and a const version returning `const std::string&` so const playlists can be read but not modified. This implementation throws `std::out_of_range` for invalid indices, including negative values.

Time rolls over 59 seconds to the next minute and 59 minutes to the next hour. This sample uses a 24-hour clock and wraps 23:59:59 to 00:00:00.

Alternative: `at(i)` can throw `std::out_of_range`; unchecked `operator[]` behaves like built-in array indexing and is faster by a small constant factor but unsafe for invalid indices. Both are O(1) per access; checked access adds a bounds comparison. Record actual 10,000,000-operation timings under -O0 and -O2 on the target machine.

C++17 Source Code

```
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
class Time {
    int h, m, s;
    void tick() {
        if (++s == 60) { s = 0; if (++m == 60) {
            m = 0; h = (h + 1) % 24;
        }}
    }
public:
    Time(int hh=0, int mm=0, int ss=0) : h(hh), m(mm), s(ss) {}
    Time& operator++() { tick(); return *this; }       // prefix
    Time operator++(int) { Time old = *this; tick(); return old; } // postfix
    bool operator==(const Time& t) const {
        return h==t.h && m==t.m && s==t.s;
    }
    bool operator<(const Time& t) const {
        if (h != t.h) return h < t.h;
        if (m != t.m) return m < t.m;
        return s < t.s;
    }
    friend std::ostream& operator<<(std::ostream& os, const Time& t) {
        return os << t.h << ':' << t.m << ':' << t.s;
    }
};
class Playlist {
    std::vector<std::string> tracks;
public:
    Playlist(std::initializer_list<std::string> list) : tracks(list) {}
    std::string& operator[](int i) {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw std::out_of_range("playlist index out of range");
        return tracks[i];
```

```
    }
    const std::string& operator[](int i) const {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw std::out_of_range("playlist index out of range");
        return tracks[i];
    }
};
int main() {
    Time t(0, 0, 5);
    Time a = t++;
    Time b = ++t;
    std::cout << a << ' ' << b << ' ' << t << '\\n';
    Time last(0, 59, 59);
    std::cout << ++last << '\\n';
    Playlist playlist{"Track One", "Track Two"};
    std::cout << playlist[0] << '\\n';
    try { std::cout << playlist[-1] << '\\n'; }
    catch (const std::out_of_range& e) { std::cout << e.what() << '\\n'; }
}
```

Expected Output

```
0:0:5 0:0:7 0:0:7
1:0:0
Track One
playlist index out of range
```

Q5 – Predict the Output (Counter)

Prediction: `c` starts at 5. `a = c++` stores the old value 5 and then c becomes 6. `b = ++c` increments c to 7 first and then copies 7 into b. Final output is 5 7 7. Postfix makes the copy of the old object.

C++17 Source Code

```
#include <iostream>
struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; }
    Counter operator++(int) {
        Counter old = *this;
        ++v;
        return old;
    }
};
int main() {
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\\n';
}
```

Expected Output

```
5 7 7
```

Q6 – The Rules of Operator Overloading

Objective: Implement a Text class that owns a dynamically allocated null-terminated character array.

Operators that cannot be overloaded: `::` (scope resolution), `.` (member access), `.*` (pointer-to-member access), `?:` (conditional operator) and `sizeof`. Their syntax/semantics are fundamental to the language. Overloading cannot change an operator's precedence, associativity or number of operands, and at least one operand must be a user-defined type.

`operator=` must be a member function. `+`, comparisons and `<<` may be non-members; `<<` is naturally non-member because the left operand is an output stream. A friend declaration is appropriate when a non-member operator needs private access, but public accessors can avoid friendship.

Copy count: `a + b + c` groups left-to-right as `(a + b) + c`. Each concatenation allocates a result and copies its characters. Return-value optimisation/move semantics may reduce object copies, but the characters still need to be assembled. For lengths n1, n2, n3, the two concatenations copy approximately (n1+n2) + (n1+n2+n3) characters, O(n1+n2+n3) for this three-term expression, with temporary storage for intermediate results.

Using std::string is normally preferred because it already manages memory safely. A custom Text class is useful for learning ownership, interoperability constraints, or specialised storage/encoding requirements. Compare against std::string and run under AddressSanitizer on the student's machine.

C++17 Source Code

```
#include <iostream>
#include <cstring>
class Text {
    char* data;
public:
    Text(const char* s = "") {
        std::size_t n = std::strlen(s);
        data = new char[n + 1];
        std::memcpy(data, s, n + 1);
    }
    Text(const Text& other) {
        std::size_t n = std::strlen(other.data);
        data = new char[n + 1];
        std::memcpy(data, other.data, n + 1);
    }
    Text& operator=(const Text& other) {
        if (this == &other) return *this;
        std::size_t n = std::strlen(other.data);
        char* fresh = new char[n + 1];
        std::memcpy(fresh, other.data, n + 1);
        delete[] data;
        data = fresh;
        return *this;
    }
    ~Text() { delete[] data; }
    Text operator+(const Text& other) const {
        std::size_t a = std::strlen(data), b = std::strlen(other.data);
        char* joined = new char[a + b + 1];
        std::memcpy(joined, data, a);
        std::memcpy(joined + a, other.data, b + 1);
        Text result(joined);
        delete[] joined;
```

```
        return result;
    }
    bool operator==(const Text& other) const {
        return std::strcmp(data, other.data) == 0;
    }
    bool operator!=(const Text& other) const { return !(*this == other); }
    friend std::ostream& operator<<(std::ostream& os, const Text& t) {
        return os << t.data;
    }
};
int main() {
    Text a("Hello "), b("C++"), c(" World");
    Text result = a + b + c;
    std::cout << result << '\\n';
    Text empty;
    Text copy = empty;
    copy = copy; // self-assignment
    std::cout << "empty=[" << copy << "]\\n";
    std::cout << std::boolalpha << (a != b) << '\\n';
}
```

Expected Output

```
Hello C++ World
empty=[]
true
```

Q7 – Design a Complex-Number Class from First Principles

Stage 1 – Requirements: Store real and imaginary components as doubles. Support default construction, construction from a real value, and construction from real plus imaginary parts. Required operations are +, -, *, /, ==, stream output and magnitude. Division by zero impedance must be detected and reported safely.

Stage 2 – Design and formulas: (a+bi)+(c+di)=(a+c)+(b+d)i; subtraction=(a-c)+(b-d)i; multiplication=(ac-bd)+(ad+bc)i; division=((ac+bd)/(c²+d²))+((bc-ad)/(c²+d²))i. Magnitude is sqrt(a²+b²). Arithmetic and equality are non-member friends in this example so scalar-to-Complex conversions can work symmetrically; output is also a friend non-member.

Stage 3 – Verification: Use R=10 Ω, XL=5 Ω and XC=2 Ω in series. Total impedance is 10 + j(5−2) = 10+j3 Ω; magnitude is sqrt(109) ≈ 10.4403 Ω. For a parallel example, Z1=10+j0 and Z2=0+j10; Z1×Z2/(Z1+Z2) = 5+j5 Ω, magnitude ≈ 7.0711 Ω. Division by 0+j0 throws a domain_error.

Stage 4 – Evaluation: std::complex<double> already implements complex arithmetic and is generally preferable for production code. A custom class demonstrates operator design and allows domain-specific checks/formatting. Both store two doubles and perform each basic operation in O(1) time and O(1) space. Floating-point equality is exact in this simple example; a real engineering application should use a tolerance appropriate to its precision requirements.

Remaining limitation: no unit tracking or configurable numerical tolerance is included. The sample uses a simple absolute near-zero denominator guard.

C++17 Source Code

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}
    Complex(double real) : re(real), im(0) {}
    Complex(double real, double imag) : re(real), im(imag) {}
    double magnitude() const { return std::sqrt(re*re + im*im); }
    friend Complex operator+(const Complex& a, const Complex& b) {
        return {a.re+b.re, a.im+b.im};
    }
    friend Complex operator-(const Complex& a, const Complex& b) {
        return {a.re-b.re, a.im-b.im};
    }
    friend Complex operator*(const Complex& a, const Complex& b) {
        return {a.re*b.re-a.im*b.im, a.re*b.im+a.im*b.re};
    }
    friend Complex operator/(const Complex& a, const Complex& b) {
        double den = b.re*b.re + b.im*b.im;
        if (den < 1e-12) throw std::domain_error("division by zero impedance");
        return {(a.re*b.re+a.im*b.im)/den,
                (a.im*b.re-a.re*b.im)/den};
    }
    friend bool operator==(const Complex& a, const Complex& b) {
        return a.re == b.re && a.im == b.im;
    }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
```

```
        os << z.re << (z.im < 0 ? " - j" : " + j") << std::abs(z.im);
        return os;
    }
};
int main() {
    try {
        const double R = 10.0, XL = 5.0, XC = 2.0;
        Complex resistor(R, 0), inductor(0, XL), capacitor(0, -XC);
        Complex series = resistor + inductor + capacitor;
        std::cout << "Series impedance: " << series
                  << ", magnitude: " << series.magnitude() << '\\n';
        Complex z1(10, 0), z2(0, 10);
        Complex parallel = (z1 * z2) / (z1 + z2);
        std::cout << "Parallel impedance: " << parallel
                  << ", magnitude: " << parallel.magnitude() << '\\n';
        Complex zero;
        Complex invalid = z1 / zero;
        std::cout << invalid << '\\n';
    } catch (const std::domain_error& e) {
        std::cout << "Error: " << e.what() << '\\n';
    }
}
```

Expected Output

```
Series impedance: 10 + j3, magnitude: 10.4403
Parallel impedance: 5 + j5, magnitude: 7.07107
Error: division by zero impedance
```